Are LLM Written Unit Tests Flaky?

Progress Report 2, ECE 382V Software Testing in the Era of Nondeterminism, Fall 2026

Shaunak Kapur (sk62288)
The University of Texas at Austin
Austin, Texas, USA

Manas Pathak (map7848)
The University of Texas at Austin
Austin, Texas, USA

1 SUMMARY

The main problem after Progress Report 1 was that we had only 3 flaky LLM tests to study. We changed the plan to attack that shortage directly.

(1) **Known flaky code (E2).** We took flaky tests that Gruber et al. [2] labeled by hand, reproduced 21 of 58 on our own runner, and asked four LLMs to write tests for the same code without seeing the flaky test (Section 4.2).
(2) **More runs (E1).** We reran all 96 suites of the 12 library study, 600 times per generated test and 200 times per human test (Section 4.1).
(3) **A second model family.** We added two OpenAI models, GPT 5.4 mini and GPT 4.1 mini.
(4) **Why (E3).** We measured how each condition builds its tests, and tested one explanation directly (Section 4.4).

We now have 39 flaky LLM tests that a developer would keep, plus 13 that a one run filter would reject (Table 1). Each has a root cause label. The main result is that one model, GPT 4.1 mini, wrote most of them. On code where randomness already causes a flaky human test, it wrote kept flaky tests in 6 of 34 suites, while Claude Sonnet and GPT 5.4 mini wrote none (Section 4.2). We also found the first order dependent LLM tests, and showed that one missing reset causes all 10 of them in E1.

Table 1: Flaky LLM tests we can now study. Kept means a one run generate and filter step would keep the test.

Source	Kept	Rejected
E1, large rerun campaign	15	n/a
E2, matched study, plain prompt	15	10
E2, mitigation trial, guard prompt	9	3
Total	39	13

Two of the 3 tests from Progress Report 1 flipped again and are inside the 15 of E1. The third, a retry loop with a 120 ms deadline, did not flip in its 600 new runs. E1 counts rejected tests as not studied, so that column is empty.

2 RESPONSE TO THE PROGRESS REPORT 1 FEEDBACK

Table 2 answers each comment.

3 STUDY DESIGN

Table 3 defines every condition. The *feature prompt* names the time, randomness, thread, and file features of each library so the model tests them. The *TestPilot style* prompt shows only signatures, docstrings, and README examples, following the TestPilot generator [7]. The *plain* prompt only asks for pytest unit tests for the given source. Every prompt is in `prompts/README.md`, word for word.

How suites were generated. Claude suites were written by Claude Code subagents, which read the source files themselves and were told not to open any test file. OpenAI suites came from the API, with the same source files pasted into the prompt, so those models never saw a test file. No generated test reuses the name of the project's flaky human test. The two pipelines differ in more than the model, which Section 4.6 discusses.

Flaky. A test is flaky if it both passed and failed on the same machine. Every run starts a fresh pytest process. Runs shuffle the test order (with the pytest-randomly plugin) and change the hash seed. Some runs also change the time zone or add CPU and memory load with stress-ng, using the configurations of Shaker [8], a tool that finds flaky tests by loading the machine. A generated test counts only if a one run generate and filter step would keep it, as in TestPilot, because that is what a developer would keep. A *test* is one pytest test case, so each parametrized case counts.

Three parts.

E1, 12 libraries. Twelve Python libraries, each with its human suite and one suite per LLM condition (96 suites).
E2, matched study. Projects where Gruber et al. found a flaky test. We reproduce that test, then ask each model to test the same code without showing it any test file.
E3, why. A static comparison of how each condition builds its tests, plus one direct experiment.

We oversample stratum 1 on purpose, because nondeterminism enters through those modules. To be clear about the order of events, our 12 libraries were chosen during the pilot, before the funnel existed. We wrote the funnel after the proposal feedback so the choice can be checked and repeated. All 12 pass every rule. Ten are in stratum 1, and two, tinydb and tabulate, are in stratum 2 as controls. The 5 packages that passed the run check are the pool for new subjects.

4 RESULTS SINCE PROGRESS REPORT 1

4.1 E1, the large rerun campaign

All E1 suites ran on GitHub Actions Linux runners with 2 virtual CPUs. Each generated test ran 300 times with a shuffled order and hash seed, 150 times with a random time zone, and 150 times under stress-ng (600 runs). Human suites are about five times larger, so each human test ran 100, 50, and 50 times (200 runs). This gives us more chances to catch a flip in an LLM test than in a human test, so the run budget favors finding LLM flaky tests. Table 5 has

Table 2: Each comment on Progress Report 1 and what we changed

Comment	What we did
The subject filter numbers were packed into one sentence. What does "and we say so" mean?	The funnel is now a table (Table 4). The phrase meant that we oversample the first stratum on purpose. We now say that directly and give the reason below the table.
Why randomly sample, and why not run more?	The random sample was only for the last funnel rule, which needs a clone, an install, and a test run per package, so we checked 6 packages per stratum to estimate the pass rate. It did not pick our subjects. We now run far more (600 runs per generated test instead of 70 to 90), and instead of adding random libraries we added projects that are known to have flaky tests.
Why not use Python flaky tests that prior datasets already found?	We now do. We reproduced 21 of the 58 flaky tests that Gruber et al. labeled by hand and that are not order dependent and not caused by the network, and used 17 of them for the matched study (Section 4.2).
Can a study rest on three flaky tests?	No, so we built E2 to find more. We now have 39 kept flaky LLM tests and 13 rejected ones (Table 1). They still cluster in a few suites, so we also report results per suite and per project.
Are LLMs just good at avoiding flakiness, and why?	It depends on the model. Sonnet and GPT 5.4 mini wrote no kept flaky test even for flaky code, and GPT 4.1 mini did. Section 4.4 shows why. LLM tests rarely share state between tests, while GPT 4.1 mini changes private library state without resetting it and rarely controls randomness. Adding the missing reset removed all 10 of its flaky E1 tests.
Why only one shot Haiku?	Haiku was only the first model where we saw flips. We now compare four models from two families under the same plain prompt. Haiku is in the middle (3 kept flaky tests of 806 in E2).
72 LLM suites does not equal 36 plus 24.	This was our error. There were 60 LLM suites (36 on October 1 and 24 on October 22) plus 12 human suites, so 72 suites in all. With the 24 OpenAI suites added since, E1 has 84 LLM suites and 96 suites in all.
What are AG-S, OS-S, OS-H?	Every condition is now defined in Table 3 before it is used.
The next steps did not address the shortage of flaky tests.	E2 was the first step. The plan for the rest of the semester grows it, with a priority order and a fallback (Section 7).

Table 3: Conditions. All generation is one shot with no test execution, except AG-S.

Code	Model	Prompt
Human	library developers	none
AG-S	Claude Sonnet, agent that runs and fixes its tests	feature
OS-S	Claude Sonnet, one shot	feature
OS-H	Claude Haiku 4.5, one shot	feature
TP-S	Claude Sonnet, one shot	TestPilot style
PL-S	Claude Sonnet, one shot	plain
PL-G5	GPT 5.4 mini (OpenAI), one shot	plain
PL-G4	GPT 4.1 mini (OpenAI), one shot	plain

Table 4: Subject funnel (top 5,000 PyPI packages)

Rule	Left
Top 5,000 by downloads (last 30 days)	5,000
Is a library (no stubs, pytest plugins)	4,701
Links a GitHub repository	4,131
Ships a pure Python wheel	3,296
At most 3 required dependencies, runs on Python 3.12	2,188
Repository has test files	1,959
Stratum 1, imports time, random, threading, asyncio, ...	1,135
Stratum 2, imports none of them	824
Suite passes offline in under 60 s (sample of 6 per stratum)	5 of 12

the results. In all, 6.3 million test executions ran, counting tests the filter rejects, which still run inside their suite.

Table 5: E1 campaign. Generated tests are those the one run filter keeps. Human and AG-S rows count every test (AG-S already ran and fixed its own suite). Flaky % has a 95% Wilson interval.

Condition	Tests	Runs each	Flaky	Flaky % (95% CI)
Human	5,587	200	5	0.09 (0.04 to 0.21)
AG-S	1,511	600	1	0.07 (0.01 to 0.37)
OS-S	1,406	600	0	0.00 (0.00 to 0.27)
OS-H	1,595	600	3	0.19 (0.06 to 0.55)
TP-S	913	600	0	0.00 (0.00 to 0.42)
PL-S	1,191	600	0	0.00 (0.00 to 0.32)
PL-G5	852	600	1	0.12 (0.02 to 0.66)
PL-G4	900	600	10	1.11 (0.60 to 2.03)

- **We cannot tell most LLM conditions apart from the human tests.** Without PL-G4, 5 of 7,468 LLM tests flipped (0.07%), against 5 of 5,587 human tests (0.09%). The intervals overlap almost completely. This does not show the two are equal, only that any difference is small.
- **PL-G4 is the outlier, but from one cause.** Its 10 flaky tests are 10 cases of three humanize test functions, they fail in 46 to 50% of runs, and all depend on test order (Section 4.4). They are one bug repeated, not ten independent findings.
- **The other LLM flaky tests are rare.** Two tenacity tests (AG-S and PL-G5) compare clock arithmetic with exact floating point equality and failed 6 and 2 times in 600 runs. Three OS-H tests depend on the time (Section 4.3).

The five human flaky tests are two arrow locale tests that depend on test order, and three pybreaker tests that flipped only under

stress-ng. Two of those are thread races, and one is a timing test. PL-G4 also lost 2 of its 12 suites (167 test functions) to files that do not import. It imported names that the packages do not export (`QueryInstance` from `tinydb`, and `CircuitBreakersState`, which does not exist, from `pybreaker`).

4.2 E2, the matched study

Reproducing known flaky tests. Gruber et al. labeled 100 flaky Python tests by hand. 42 are caused by the network, which leaves 58. For each, we checked out the project at the recorded commit, installed it in a fresh Python 3.8 environment, and ran the flaky test up to 150 times. Table 6 shows what happened.

Table 6: Reproducing the hand labeled flaky tests of Gruber et al.

Outcome	Tests
Hand labeled, not order dependent, not network	58
Repository gone or install failed	3
Always failed (old dependencies, missing data files)	21
Never failed in 150 runs	13
Reproduced (passed and failed)	21
Used for the matched study	17

We left out 4 of the 21. Two fail because of downloads or a web API (wiki-futures, ruTS). The other two are Hypothesis property tests that mostly hit the time limit on our runner (ashpool, humansort). Of the 17 projects we used, 16 are labeled randomness and one async wait. So E2 tests code where randomness causes flaky tests, not flaky code in general.

Generating tests for the same code. For each project we found the code the flaky test exercises (`scripts/matched_targets.py`, then checked by hand). We asked each model for 10 to 25 tests of that code, two independent samples per model. Every generated suite then ran 200 times on the same runner, half in file order and half in shuffled order. The known flaky human test ran 200 times next to it as a control. It failed at least once in 16 of 17 projects (384 failures in 3,400 runs). The exception is data-genie, which failed in 98 of 150 runs during reproduction but never in the matched run. We are still checking why.

Table 7: E2 by model, 17 projects, 34 suites per model, plain prompt, 200 runs per test

Model	Kept	Flaky kept	Suites with one	Flaky rejected
Claude Sonnet	789	0	0	0
Claude Haiku 4.5	806	3	2	0
GPT 5.4 mini	429	0	0	0
GPT 4.1 mini	558	12	6	10

Table 7 has the results. "Flaky rejected" counts tests that were flaky but failed in the first run, so a one run filter would have dropped them. The models wrote 792, 810, 554, and 746 tests in that order, and the filter removed the rest.

Flaky tests cluster within suites, so we compare at two levels.

- **Per test,** GPT 4.1 mini (12 of 558, 95% interval 1.2 to 3.7%) is flakier than Sonnet ($p < 0.001$, Fisher's exact test), GPT 5.4 mini ($p = 0.002$), and Haiku ($p = 0.003$). This treats tests as independent, which they are not.
- **Per suite,** GPT 4.1 mini has a kept flaky test in 6 of 34 suites, against 0 of 34 for Sonnet and GPT 5.4 mini ($p = 0.025$). Per project it is 5 of 17 against 0 of 17 ($p = 0.044$). Against Haiku (2 of 34 suites) the difference is not significant ($p = 0.26$).

Two of the 12 GPT 4.1 mini tests flip only because another test in the same suite always fails and leaves a fake behind (Section 4.3). A one run filter would remove that test. Without these two, the per suite comparison with Sonnet gives $p = 0.053$. So the evidence that GPT 4.1 mini writes more flaky tests is clear per test and borderline per suite, and growing E2 is the way to settle it (Section 7).

In 10 of the 16 projects whose human test failed in our matched runs, no model wrote a single kept flaky test. This comparison is about tests for the same code, not about rates. The human test was chosen because it is flaky, so the human side is flaky by construction. E2 measures how often a model writes a flaky test when the code under test makes that easy.

4.3 Root causes

We labeled the root cause of every new flaky test with the categories of Luo et al. [4] and Gruber et al., from the test, the code it calls, and a reproduction of the failure where possible. A second, independent check of every label agreed with it. Table 8 has the counts, and `results/root_causes/` has every label with its evidence.

Table 8: Root causes of the 52 flaky LLM tests

	E1	E2 plain		E2 guard	
Root cause	kept	kept	rej.	kept	rej.
Randomness, unseeded generator	0	10	9	4	2
Randomness, salted `hash()`	0	3	1	5	1
Test order dependency	10	2	0	0	0
Time	3	0	0	0	0
Floating point on a clock value	2	0	0	0	0
Total	15	15	10	9	3

E2 tests inherit the nondeterminism of the code. Most randomness cases call code that draws from an unseeded global generator (NumPy, `random`, or PyTorch) and then assert one particular outcome. In the two torch-layer-normalization cases the test itself draws unseeded input. In all but one E2 case (a gamble deck shuffle test), the randomness is the same that makes the project's human flaky test flip. In cuckoopy the source is less obvious. The filter places items by Python's built in `hash()` of a string, which changes with `PYTHONHASHSEED` in every process, and the tests insert keys that share a fingerprint. Seeding `random` cannot fix this.

A second order dependent pattern. Besides the PL-G4 humanize tests (Section 4.4), two GPT 4.1 mini tests in pylabeledrf

depend on order. A third generated test, the polluter, replaces a module level function with a fake and always fails before the line that restores it, so the fake stays for the rest of the process. The two victims fail whenever they run after it.

E1 timing tests. The AG-S and PL-G5 tenacity tests build a fake timestamp as `start + 5.0` (or `+ 1`) from a real `time.monotonic()` reading and assert exact equality with the elapsed time. For some values of `start` the subtraction does not give back exactly 5.0, so the result depends on the clock value. The OS-H `test_until_with_time_object` builds a clock time one hour ahead, which wraps past midnight when the run starts between 23:00 and 24:00 local time. That is why it failed only in the time zone mode. The other two OS-H tests are the Progress Report 1 cases (a time of day string from the real clock, and a jittered wait under a lower time bound).

One plugin interaction. One guard prompt test in pylabeledrf seeds `random` in a fixture, but pytest-randomly seeds it again just before the test body in the shuffled runs. The test fails for most seeds, so we count it as randomness.

The human pybreaker timing test stores its open time in Redis as whole seconds, so a one second timeout can already be over when it is checked.

4.4 E3, why most LLM tests do not flip

`scripts/characterize.py` scans every test function in E1 for shared state, isolation helpers, clock and sleep use, timing bounds, and randomness. Table 9 shows the features most tied to order dependence for the plain prompt conditions. All conditions are in `results/why_analysis.md`.

Table 9: Share (%) of E1 test functions with each feature

Feature	Human	PL-S	PL-G5	PL-G4
Test functions	3,077	927	897	985
Any shared state between tests	23.9	1.0	6.5	1.2
Class or module scoped fixture	19.5	0.0	0.0	0.0
Touches a private attribute	12.3	0.9	14.2	15.5
Resets library state after the test	0.3	8.6	9.8	0.0

Why most LLM tests do not flip. Human suites share setup through class and module scoped fixtures (19.5% of human test functions), and 4.2% have setup code with no matching teardown. The 4 order dependent human flaky tests from Progress Report 1 all sit on such shared state. No LLM condition uses a fixture wider than one test or has setup without teardown, and in every Claude condition 35 to 48% of tests build a fresh library object, against 18.5% of human tests.

Why GPT 4.1 mini flips. The GPT suites touch private attributes as often as human suites. Touching private state is a risk, not a cause by itself. PL-G5 does it as often but never patches the humanize translation state, and it has one flaky test. PL-G4 does patch it. Three of its humanize test functions (10 test cases) call `monkeypatch.setattr(i18n._CURRENT, "locale", ...)`. `_CURRENT` is a `threading.local`, and its `locale` attribute exists only after some earlier test has called `activate` or `deactivate`. So these tests pass when such a test ran earlier and fail when none has. The one run filter ran the file in its written order, where they pass, so it kept them. PL-S resets this state around every humanize test, and PL-G4 never does.

A direct test. We ran the PL-G4 humanize suite 50 times in shuffled order as generated, and 50 times with one added fixture that calls `deactivate()` before and after every test (`scripts/humanize_reset_check.py`). Without the fixture 10 tests flipped. With it none did. The 4 tests that always fail are the same in both runs.

Randomness in E2. In E2 the code under test draws random numbers itself. Sonnet seeds a generator in 14 of its 34 suites, Haiku in 13, GPT 5.4 mini in 5, and GPT 4.1 mini in 3. GPT 5.4 mini seeds rarely but still wrote no flaky test, so seeding is not the whole answer. In spreading_dye_sampler, for example, Sonnet seeds NumPy and often passes `squeeze=True`, which avoids the random path that returns `None`. GPT 5.4 mini replaces the random draws with fakes through `monkeypatch`. GPT 4.1 mini rarely does either, and several of its tests assert that a random walk always returns a result.

Why Haiku flipped in Progress Report 1. With the same prompt as one shot Sonnet, Haiku wrote about twice as many test functions (1,750 against 907). It never froze the clock in schedule (0 of 182 tests, against 62 of 78 for Sonnet), never injected a fake sleep in tenacity, and wrote one sided timing bounds with less headroom. Its flaky tests came from those timing habits.

4.5 First mitigation trial

We started the mitigation step early on the model that needed it. The *guard prompt* adds one paragraph to the plain prompt. It asks for tests that give the same result in every run and order, and lists the usual causes (unseeded randomness, wall clock time, test order and global state, set order, exact floating point). We regenerated all 34 GPT 4.1 mini suites of E2 with it and ran them the same way.

The prompt helped only a little. GPT 4.1 mini now seeds in 10 of 34 suites instead of 3. Kept flaky tests disappeared from 3 of the 5 projects that had them (scikit-procrustes, spreading_dye_sampler, torch-layer-normalization). But 5 new kept flaky tests appeared in cuckoopy, where seeding `random` cannot help because the cause is `hash()`. Overall 9 of 550 kept tests are flaky, against 12 of 558 ($p = 0.66$), and the filter now rejects more tests (221 of 771, against 188 of 746).

We also estimated what a stricter filter would catch, from the measured failure rate of the 39 kept flaky LLM tests. If a test fails in a fraction f of runs, k extra runs catch it with probability $1-(1-f)^k$. Table 10 shows the expected share caught. Twelve of the 39 fail in under 5% of runs, so a short rerun filter misses them. The order dependent tests are caught only if the extra runs also shuffle the order. Stable tests cost only run time, because they never fail.

Table 10: Expected share of the 39 kept flaky LLM tests that k extra filter runs would catch

Extra runs k	1	3	5	10
Caught (%)	26	52	63	73

4.6 Threats to validity

- **Pipeline, not only model.** Claude suites came from Claude Code subagents that read files, and OpenAI suites came from the API with sources pasted in. Sampling settings and system prompts differ too. The model comparison is therefore a comparison of two pipelines with four models.
- **Scope of E2.** 16 of 17 projects are randomness projects, so E2 says nothing yet about time or concurrency.
- **Clustering.** Flaky tests cluster within suites and projects, so per test p values overstate the evidence. We report per suite results next to them.
- **Run order record.** The matched runner stores totals per test, not which half (file order or shuffled) each failure came from.

5 MILESTONE GOALS AND STATUS

Table 11 lists the Intermediate Steps 2 goals from the proposal and the next steps we promised in Progress Report 1.

6 CHALLENGES

- **Old projects are hard to run.** Only 21 of 58 known flaky tests reproduced. Most of the rest always failed because of old dependencies or missing data files. We added a step that installs any missing imported package and tries again, which made Intervallum run, for example.
- **A bug cost us one project for a while.** fuzzylogic cannot be installed, and its own tests add its `src` folder to the import path by hand. Our runner did not, so all 8 generated fuzzylogic suites failed to import. We fixed the runner and reran fuzzylogic before writing this report.
- **Models invent names.** PL-G4 lost two whole E1 suites to imports of names the packages do not export.
- **A one run filter in file order hides order dependence.** The PL-G4 humanize tests always pass in file order, so they were kept.
- **Flakiness is still rare for most models.** Sonnet and GPT 5.4 mini produced no kept flaky test in E2, so their behavior can only be bounded, not studied from flips.

7 NEXT STEPS AND RECOVERY PLAN

The plan grows E2, because it is the part of the study that produces flaky LLM tests. The steps are in priority order. Progress Report 3 is due November 20.

(1) **More known flaky projects (Shaunak, by November 13).** Gruber et al. found 952 flaky tests that are not order dependent in 277 projects, without hand labels. We have tried 58 tests in 58 projects. We will reproduce one test from each of the roughly 220 remaining projects on GitHub Actions and drop network failures by their error messages. At our rates so far (21 in 58 reproduced, 17 in 58 usable) this should give 60 to 80 projects, and we will stop at 40 new ones. We will take time, concurrency, and IO projects first, so E2 is not only about randomness. The runner will also record each run's order, so file order and shuffled order can be compared.

(2) **Root causes for new flaky tests (Manas, ongoing).** Label every new flaky test the same way as above.

(3) **Mitigation, research question 4 of the proposal (Shaunak, by November 20).** Run the guard prompt for all four models. Add an automatic fixture that seeds `random`, NumPy, and PyTorch and resets known library state before every generated test. Compare a 3 and 5 run filter that shuffles the order. For each we report flaky tests kept, good tests lost, coverage, and cost.

(4) **An order dependent arm (Manas, by November 20).** Gruber et al. also found 4,461 order dependent tests in 615 projects. For 10 of these projects we will generate LLM tests for the same code and run them inside the human suite in shuffled orders, to test directly whether LLM tests avoid shared state or become victims and polluters.

(5) **Delay injection (if time allows).** Use FlakeRake [6], which inserts delays to reproduce timing failures, on the E1 tests with timing bounds.

Check point and fallback. On November 13 we check how many new projects reproduced. If fewer than 20 did, we stop growing E2 and move that time to steps 3 and 4. Step 5 is the first to cut. If the strong models still write almost no flaky tests, the final report will give upper bounds per model and use E3 to explain them, as planned in the proposal.

Division of work. Since Progress Report 1, Manas did the literature review and the root cause labeling of the new flaky tests. Shaunak built the reproduction and matched study pipeline, generated the OpenAI suites, ran the E1 campaign and the mitigation trial, and wrote the analysis scripts and this report. The split for the next milestone is in the list above. Manas will also write the related work section of the final report.

8 RELATED WORK

Luo et al. [4] give the root cause categories we use, and Gruber et al. [2] show what flakiness looks like in Python and provide the labeled tests behind E2. TestPilot [7] is the generate and filter pipeline we imitate. It keeps a test after one passing run, which Section 4.5 shows is not enough for flaky tests. Shaker [8] gives our stress-ng configurations, iDFlakies [3] the shuffled order reruns, and FlakeRake [6] the delay injection we plan. Parry et al. [5] survey the field. None of these studies looks at tests written by LLMs.

9 CODE, DATA, AND DOCUMENTATION

Everything is in the public repository https://github.com/shaunak1206/ece382v-flaky-llm-tests.

- `README.md` explains how to set up the environment and gives the command for every experiment.
- `docs/DATA.md` documents every result file and field. Examples are the per test `counts` and `kind` in each rerun file, the columns of `results/gruber/candidates.csv`, and the fields of the matched study files.
- Raw results are under `results/`. Generated suites are under `gen_*`, `llm_tests/`, and `matched/`. Every prompt is in `prompts/README.md`. Root cause labels are in `results/root_causes/`.

New since Progress Report 1:

Table 11: Goals for this milestone

Goal	Status	Notes
Rerun every flagged test at least 1,000 times on the GitHub runners, with and without CPU stress	Done	Done for Progress Report 1 (over 1,300 runs each). Every E1 test now also has 600 (generated) or 200 (human) runs.
Add a non Claude model	Done	GPT 5.4 mini and GPT 4.1 mini with the plain prompt, in E1 and E2. We did not run them with the TestPilot style prompt, and spent that budget on E2 instead.
Add the plain prompt	Done	Added October 22 (PL-S), now also used for every E2 model.
Measure scanner precision and recall	Done	Done for Progress Report 1 on 96 hand labeled tests of Gruber et al. Recall is low (10 of 96), because most of the nondeterminism is inside the library, not in the test. It flagged 35 of 714 tests that were never reported flaky (4.9%).
Measure coverage of every suite	Done	Mean line coverage is 94.4% for human suites, 82.6% for PL-S, 75.6% for PL-G5, and 67.0% for PL-G4, two of whose suites did not import. We leave out filelock, whose Windows module never runs on our machines (`results/coverage/`).
Search for order dependent tests in the style of iDFlakies [3], a tool that reruns tests in many orders to find order dependence	Partly	Every rerun shuffles the order, which found the PL-G4 humanize tests and the pylabeledrf victims. Progress Report 1's mixed suites (generated tests run inside the human suite) found three more. A systematic search for the polluter of every victim is not done yet.
Add four new subjects from the funnel	Replaced	The matched study adds 17 projects with known flaky code, which attacks the shortage of flaky tests more directly.
Pick stress-ng configurations with Shaker's selection step	Not done	Only three human tests flip only under stress-ng, too few to choose configurations from.
Run FlaPy [1] (the rerun tool of Gruber et al.) on every suite	Dropped	FlaPy ran on two suites. Our harness already reruns in fresh processes in file and shuffled order, which is what FlaPy does, so we stopped there.
Start the mitigation early	Started	Guard prompt trial and the rerun filter estimate (Section 4.5).

- `scripts/gruber_reproduce.py` and the `gruber-reproduce` workflow reproduce a known flaky test at its original commit.
- `scripts/matched_targets.py` finds the code a flaky test exercises. `scripts/matched_run.py` and the `matched-study` workflow rerun generated suites next to the human flaky test. `scripts/analyze_matched.py` makes the E2 tables.
- `scripts/gen_openai.py` generates the OpenAI suites, including the guard prompt, and logs every call.
- `scripts/scale_reruns.sh`, the `scale-reruns` workflow, and `scripts/analyze_scale.py` run and summarize the E1 campaign.
- `scripts/characterize.py` measures how each test is built (`results/why_analysis.md`), and `scripts/humanize_reset_check.py` runs the reset experiment.

REFERENCES

[1] Martin Gruber and Gordon Fraser. 2023. FlaPy: Mining Flaky Python Tests at Scale. In *IEEE/ACM International Conference on Software Engineering (ICSE), Companion Proceedings, Tool Demonstrations.*

[2] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *IEEE International Conference on Software Testing, Verification and Validation (ICST).* 148–158.

[3] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *IEEE International Conference on Software Testing, Verification and Validation (ICST).* 312–322.

[4] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE).* 643–653.

[5] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM Transactions on Software Engineering and Methodology* 31, 1 (2021).

[6] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *IEEE International Conference on Software Testing, Verification and Validation (ICST).*

[7] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.

[8] Denini Silva, Leopoldo Teixeira, and Marcelo d'Amorim. 2020. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *IEEE International Conference on Software Maintenance and Evolution (ICSME).*